

4.6 Translations for Active Record Models

You can use the
methods `Model.model_name.human` and `Model.human_attribute_name(attribute)` to transparently look up translations for your model and attribute names.
For example when you add the following translations:

```
en:
  activerecord:
    models:
      user: Dude
    attributes:
      user:
        login: "Handle"
      # will translate User attribute "login" as "Handle"
```

Then `User.model_name.human` will return "Dude" and `User.human_attribute_name("login")` will return "Handle".
You can also set a plural form for model names, adding as following:

```
en:
  activerecord:
    models:
      user:
        one: Dude
        other: Dudes
```

Then `User.model_name.human(count: 2)` will return "Dudes". With `count: 1` or without params will return "Dude".
In the event you need to access nested attributes within a given model, you should nest these under `model/attribute` at the model level of your translation file:

```
en:
  activerecord:
    attributes:
      user/gender:
        female: "Female"
        male: "Male"
```

Then `User.human_attribute_name("gender.female")` will return "Female".

4.6.1 Error Message Scopes

Active Record validation error messages can also be translated easily. Active Record gives you a couple of namespaces where you can place your message translations in order to provide different messages and translation for certain models, attributes, and/or validations. It also transparently takes single table inheritance into account.

This gives you quite powerful means to flexibly adjust your messages to your application's needs.